



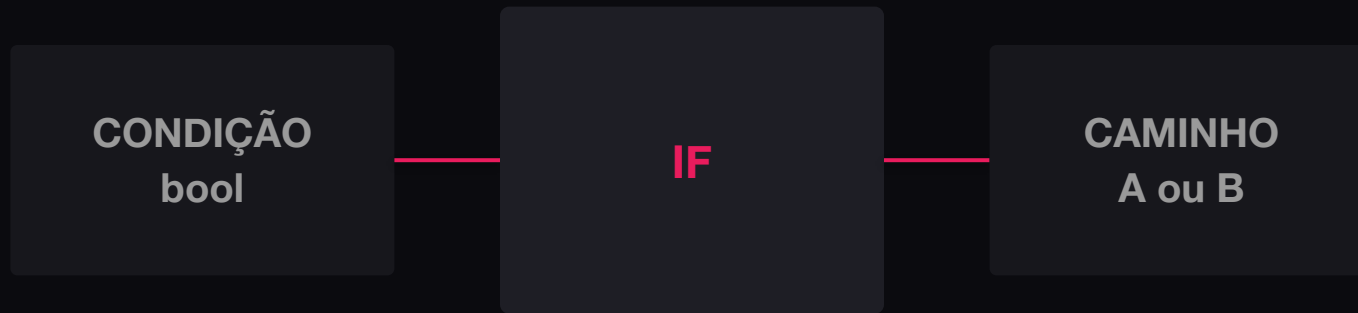
DART • IF

Dominando o If

Como o programa escolhe caminhos diferentes com base em uma condição.

O QUE É UM IF?

Uma estrutura de decisão: o programa testa uma condição booleana e escolhe qual bloco de código executar.



- Direciona o fluxo do programa com base em uma regra.
- Permite validar dados e liberar (ou negar) um acesso.
- É a base de toda tomada de decisão em código.

A ANATOMIA BÁSICA

```
if (idade >= 18) {  
    ...  
} else {  
    ...  
}
```

if (condição) — O Teste

Uma expressão que resulta em true ou false.

{ } — Bloco Verdadeiro

Executado somente quando a condição é true.

else — A Alternativa

Executado quando a condição é false. É opcional.

() — Operadores

Relacionais e lógicos combinados formam a condição.

...

ENCADEANDO CONDIÇÕES: ELSE IF

Quando há mais de duas possibilidades, encadeie condições em ordem.

```
if (nota >= 9) {  
    print('Excelente');  
} else if (nota >= 7) {  
    print('Bom desempenho');  
} else if (nota >= 6) {  
    print('Recuperação');  
} else {  
    print('Reprovado');  
}
```

Cada else if testa uma nova condição só quando a anterior for falsa — e a ordem em que você escreve importa.

OPERADORES RELACIONAIS E LÓGICOS

São eles que constroem a condição avaliada pelo if.

```
> maior que  
< menor que  
>= maior ou igual  
<= menor ou igual  
== igual a  
!= diferente de
```

```
&& E (ambas verdadeiras)  
|| OU (ao menos uma)  
! NÃO (inverte o valor)
```

```
usuarioAtivo && temAssinatura
```

Relacionais comparam dois valores e resultam em bool. Lógicos combinam ou negam condições booleanas.

O ERRO CLÁSSICO: = VS ==

Um único sinal de igual muda completamente o significado do código.

```
if (ativo = true) {  
    // ERRO: atribuição  
}
```

```
if (ativo == true) {  
    // certo: comparação  
}
```

= atribui um valor à variável. == compara dois valores. Confundir os dois é uma fonte clássica de bugs.

FORMA COMPACTA: O TERNÁRIO

Uma versão enxuta do if/else que retorna um valor diretamente.

```
final resultado = condicao ? valorSeVerdadeiro : valorSeFalso;
```

Substitui um if/else de uma linha. Use apenas em decisões curtas — abusar do ternário prejudica a legibilidade.

EXEMPLO COMENTADO

```
void main() {  
  
    final idade = 17;  
    if (idade >= 18) {  
        print('Maior de idade');  
    } else {  
        print('Menor de idade');  
    }  
  
    final nota = 8.2;  
    if (nota >= 9) {  
        print('Excelente');  
    } else if (nota >= 7) {  
        print('Bom desempenho');  
    } else if (nota >= 6) {  
        print('Recuperação');  
    } else {  
        print('Reprovado');  
    }  
}
```

...

SAÍDA ESPERADA

```
$ Menor de idade  
$ Bom desempenho
```

idade = 17 cai no else (menor de idade). nota = 8.2 satisfaz o primeiro else if verdadeiro (≥ 7), então para ali.

IF/ELSE, TERNÁRIO OU GUARD CLAUSE?

	if / else if	Ternário ? :
Retorna um valor?	Não diretamente — executa um bloco.	Sim, é uma expressão.
Suporta várias linhas?	Sim, qualquer lógica.	Não, só uma expressão curta.
Uso ideal	Regras de negócio e decisões em cadeia.	Escolher entre dois valores simples.

MUNDO REAL: INTERFACE CONDICIONAL

O widget exibido na tela muda de acordo com uma condição.

```
Widget build(BuildContext context) {  
  if (carrinho.isEmpty) {  
    return Text('Carrinho vazio');  
  }  
  return ListaDeItens(carrinho);  
}
```

Um guard clause no início do build() evita renderizar a lista quando não há nada para mostrar.

A ARTE DO CÓDIGO LIMPO

✓ Certo

```
if (!ativo) return false;  
if (!assinante) return false;  
return true;
```

- Guard clauses: sai cedo, sem aninhar.
- Cada if testa uma única regra.
- Fácil de ler de cima para baixo.

✗ Errado

```
if (ativo) {  
    if (assinante) {  
        return true;  
    }  
}  
return false;
```

- if aninhado dentro de if.
- Regra final está escondida no fundo.
- Cada nível novo dificulta a leitura.

JUNTANDO TUDO

```
bool podeAcessarPremium(bool ativo, bool assinante) {  
    if (!ativo) return false;  
    if (!assinante) return false;  
    return true;  
}
```

Guard clauses

Evitam aninhamento: cada regra sai cedo com return.

&& (E lógico)

Exige que ambas as condições sejam verdadeiras.

Retorno antecipado

Deixa a regra final (return true) bem visível.

Uma regra por if

Cada condição testa uma única coisa, claramente.

MINI-RESUMO (CHEAT SHEET)

if / else

Decisão binária: um caminho ou outro.

else if

Encadeia condições testadas em ordem.

&& || !

Combinam ou negam condições booleanas.

condição ? a : b

Forma compacta de um if/else que retorna valor.

= vs ==

Atribuir não é o mesmo que comparar.



DIVIRTA-SE!

Profº Vinny Albuquerque
profvinny.albuquerque@fiap.com.br